

Optimizing the Optimizers: Measuring Learned Control of MIP Solvers on a Single DGX Spark

Berk Orbay

`berk.orbay@tideseed.com`*

Technical report — 24 September 2026

Abstract

The default settings of a mixed-integer programming (MIP) solver are tuned to do well on average; a user with a fixed time budget cares about the instance at hand. On one NVIDIA DGX Spark we measure, for HiGHS, SCIP and NVIDIA cuOpt, how much sooner a good solution could be found by choosing settings per instance, whether that gain comes from the initial choice or from switching settings during the solve, and what simple and learned controllers recover, with their decision time charged to the budget. The measure is the primal integral, the gap to the best known solution averaged over the budget. Over 8,000 runs on MIPLIB 2017, two ML4CO families and PGLib-UC unit commitment give three main findings. First, choosing the best of six configurations per instance would lower the primal integral by 19 % over all 336 attempted instances, 25 % on the 313 where some configuration finds a solution and 41 % on MIPLIB, measured with one seed. Second, inside SCIP most of this comes from the initial choice: the right static setting is worth 12 % (45 instances) and 20 % (the 20 with the most headroom), while the six switching schedules we tried add about 2 % at 120s and nothing detectable at 300s. Third, on held-out ML4CO instances one fixed setting improves on SCIP’s defaults by 33 %, and by 41 % when applied at the solver’s first callback instead of before presolve, which on load balancing avoids one start-up heuristic; an untrained local language-model agent makes the same first move in most runs, but the fixed setting applied at that callback beats it on 22 of 30 instances. Running cuOpt beside HiGHS or SCIP, or passing cuOpt’s solutions into SCIP, gains 14–30 % with no model. The harness is released as a Python library with the experiment definitions, and the original run trajectories as a versioned archive.

1 Introduction

Planners, schedulers and energy traders who use MIP solvers rarely wait for a proof of optimality. They need a good solution within a fixed time, and how soon a solver finds one depends strongly on its settings. Figure 1 shows an example. On the MIPLIB instance `h80x6320d`, SCIP with its default settings stays 62 % away from the best known solution for the first minute and then finds the optimum at 61s. The same solver with cutting-plane separation switched off is within 8 % after two seconds but has not reached the optimum when the 120s run ends. The primal integral

*Experiments designed, run and written up with AgentRA (Agent Research Assistant; AgentRA is mainly Claude Opus 5.5), under the author’s direction; every number was produced by the code and experiment definitions in the public repository <https://github.com/berkorbay/optopt> and can be re-derived with `optopt reproduce`. A companion file for automated readers is provided as an ancillary file and in the repository (Appendix K). The study was reviewed, with suggestions for improvement, by Claude Fable 5.1 (Anthropic) and OpenAI GPT-6 Astra (ChatGPT).

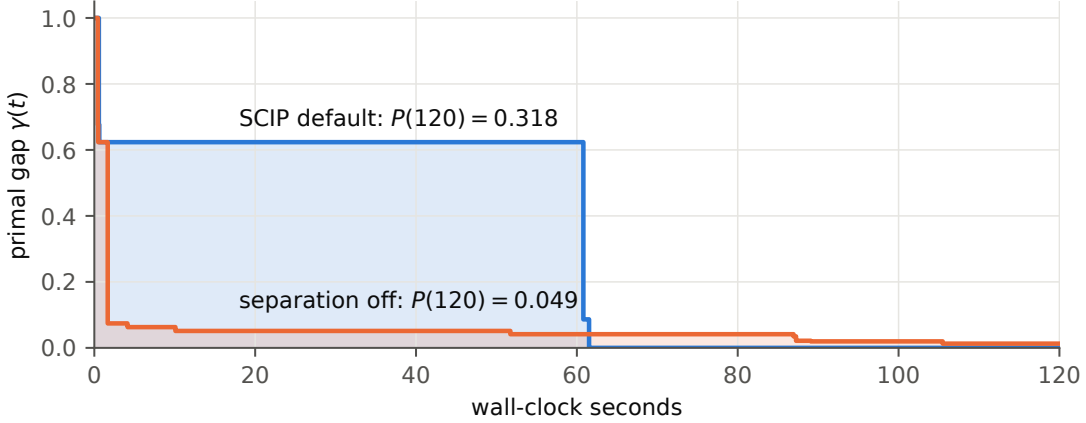


Figure 1: The primal integral on one run each of SCIP’s default and of SCIP with separation off (MIPLIB h80x6320d, 120s, seed 0). The curve is the gap between the best solution found so far and the best known solution; it is 1 until the first solution. $P(120)$ is the shaded area divided by 120s: lower is better.

$P(T)$ [4] turns such a curve into one number, the average gap over the budget T , and so rewards finding good solutions early: here 0.318 for the default and 0.049 without separation.

Two developments make such differences worth exploiting. GPU solvers such as cuOpt [42] now run beside CPU solvers on the same small machine, and agents — from small learned models to language models — can read a solver’s state and change its settings while it runs. This paper measures how much there is to gain, whether the gain comes from the initial choice of settings or from changing them later, and which controllers recover it. All measurements use the conditions a user faces: one machine, a wall-clock budget that includes the controller’s own decision time, and instances the controller has not seen.

Headroom. We call *headroom* the gain available from choosing a configuration per instance instead of using one configuration for all. A small example: configuration A scores $P = 0.2$ on instance 1 and 0.6 on instance 2; configuration B scores 0.5 and 0.1. The best single configuration is B (mean 0.30, against 0.40 for A). An *oracle* that picks the better configuration for each instance scores 0.2 and 0.1 (mean 0.15). The headroom is the oracle’s improvement over the best single configuration: $(0.30 - 0.15)/0.30 = 50\%$. It bounds what any method choosing among these configurations can gain; if one configuration were best everywhere, it would be zero. A solver’s result also varies from run to run with its random seed, and an oracle chosen and scored on the same run counts that luck as headroom. Where a second seed exists (Sections 5 and 6) the oracle therefore chooses on one seed and is scored on another. The cross-solver headroom of Section 3 uses one seed; Section 3 reports how much of it survives a second seed.

Contributions.

1. **How much there is to gain.** Choosing per instance among six configurations of HiGHS, SCIP and cuOpt would lower the primal integral by 19 % over all attempted instances (25 % where some configuration finds a solution) and 41 % on MIPLIB, compared with the best single configuration (one seed); on a five-seed subset about 80 % of the gain repeats (Section 3).

2. **Choosing versus switching.** Inside SCIP we split the gain into the part from choosing the right setting at the start and the part from changing settings during the solve (Figure 2). The initial choice is worth 12 % at 120 s (45 instances) and 20 % at 300 s (the 20 with the most headroom); the six switching schedules we tried add about 2 % and nothing detectable on the primal integral, and 1.5–1.6 % on the primal-dual integral (upper 95 % bounds 3.8–4.3 %). We have not found this split measured for MIP before (Section 5).
3. **Agents measured on equal terms.** Every controller gets the same problem, solver and wall-clock budget, and its thinking time counts against the budget: the defaults, the best fixed setting, an online bandit, Laya, and local and hosted language models with and without Laya as a tool. A control that applies a fixed setting at the same callback as the agents shows that the language-model agent’s advantage over the fixed setting comes from *when* it acts: applied at that callback, the fixed setting beats the agent on 22 of 30 instances, and no benefit was detected from the agent’s later decisions (Section 6).
4. **Two solvers together.** Running cuOpt beside HiGHS improves on HiGHS alone by 30 %, and beside SCIP on SCIP alone by 21 % (44 instances); passing cuOpt’s solutions into SCIP improves SCIP’s own primal integral by 14 %. We report which solver interfaces allow this safely (Section 7).
5. **A reusable harness.** The Python library `optopt`: solver wrappers that record every solution and bound over time, an agent interface that charges decisions to the wall clock, an exact-arithmetic solution checker, the experiment definitions, the original run trajectories as a versioned archive, and `optopt reproduce`, which re-derives every table from them. It also records a pitfall for anyone measuring SCIP: when SCIP reports a new best solution, its primal bound still holds the previous one (Appendix D).

Related work. Learning for MIP is an established field [3, 46], with learned branching [16, 29], heuristics and diving [12, 30, 41], large-neighbourhood search [24, 47, 48], cut selection [43, 50, 53], node selection [21, 33], restarts [2] and configuration [25, 35, 51]. Per-instance algorithm selection and portfolios [7, 45, 55, 56], including selection among MIP solvers [36, 49], online bandits inside or on top of SCIP [10, 13, 23], switching strategies during the solve [5, 15, 22], racing solver copies [40, 57] and language models that configure solvers [9, 34, 39, 44] all exist; we use these methods and claim no novelty for them. What we add is a measurement under a user’s conditions: a CPU and GPU portfolio scored by the primal integral, the choosing-versus-switching split for MIP (made for black-box optimisation by Vermetten et al. [52]), and very different agents compared on the same budget with their thinking time charged. Appendix A details a structured search of about 180 works, including relevant work we could not read.

2 Setup

Machine and solvers. One NVIDIA DGX Spark (GB10: 10 Cortex-X925 and 10 Cortex-A725 cores, Blackwell GPU, 121 GiB memory shared by CPU and GPU). HiGHS 1.15.1, SCIP 10.0 and cuOpt 26.08 from released wheels. Timed CPU runs use one thread. In the SCIP experiments of Sections 5–7 each run is pinned to one Cortex-X925 core with a hard memory cap; the cross-solver grid of Sections 3 and 4 was not pinned, so its runs moved between fast and slow cores (Appendix B). Budgets are equal for the solver; they do not equalise total compute: a local language-model server and cuOpt’s host threads are extra.

Instances. 80 MIPLIB 2017 benchmark instances [19], at most one per MIPLIB group; the ML4CO item-placement and load-balancing families [17], trained on their official training instances and tested on instances from their official validation split; and the 56 PGLib-UC unit-commitment cases [26], built with the benchmark’s own formulation [31]. On 23 of the 24 FERC cases no configuration finds a solution within 60 s (Appendix C).

Metric. The normalised primal integral $P(T) \in [0, 1]$ [4] (Figure 1): the relative gap between the best solution found so far and the best known value, averaged over the wall-clock budget T , with the gap set to 1 until the first solution. The budget starts once the model has been read (median 1.8 s, at most 5.8 s), except in the live two-solver runs of Section 7, where it starts when both processes are launched and so includes reading the model. The primal-dual integral (PDI) uses the solver’s own bound in place of the best known value, so it also rewards progress on the bound. The final solution of every SCIP run, and of the HiGHS and cuOpt runs in the two-solver experiments, is checked independently against the model; earlier solutions that enter $P(T)$ are not checked (Appendix D).

Protocol. Where two seeds exist, oracles choose on one seed and are scored on another; the cross-solver grid has one seed (Section 3). Learned selectors are evaluated by grouped cross-validation: ML4CO by its official split, PGLib-UC by power system; a second scheme leaves a whole family out. Paired tests are over instances, with the seeds of an instance averaged. The machine was shared with other work, and results shift between sessions: the same two arms on the same 60 runs scored 0.7–2.7 % worse on the second day than on the first. We therefore compare arms within one session and treat differences below about 3 % between sessions as noise. Within a session, arms ran in a fixed order per instance, not randomised, so slow drift within a session is not excluded. Only the 10 % headroom threshold of Section 3 was set in advance. We name choosing versus switching (Section 5) and the timing-matched control (Section 6) as the principal comparisons after the fact, to guide reading; like all others they are exploratory, not pre-registered.

3 How much is there to gain?

Six configurations — default and primal-focused variants of HiGHS, SCIP and cuOpt (Table 9) — were run on all 336 instances. Choosing per instance would lower the primal integral by 19 % over all of them and by 41 % on MIPLIB (Table 1), above the 10 % threshold we set before any run. On 23 unit-commitment cases, all from the FERC system, no configuration finds a solution within 60 s, so every configuration scores $P = 1$ there; on the 313 instances where at least one configuration finds a solution the headroom is 25 %. The selectors of Section 4 are trained and evaluated on those 313, which include one FERC case. This oracle is chosen and scored on the same seed. On a 12-instance MIPLIB subset run with five seeds and the four CPU configurations, a held-out oracle realises 26.3 % of an apparent 31.8 %, that is 83 % (Table 11); cuOpt was not re-run with extra seeds. These runs were not pinned to cores, so a run’s effective budget varied by up to a factor of two, which inflates single-seed headroom. cuOpt’s solutions are feasible at its own tolerance of 10^{-4} ; two of 140 checked exceed 10^{-6} (Appendix D). No configuration wins everywhere: HiGHS with more heuristics, or SCIP’s default, is the best fixed choice where the CPU solvers finish, while cuOpt wins at short budgets and on the MIPLIB instances that neither CPU solver finishes in 300 s (Appendix E).

family	instances				oracle headroom		
	all	some sol.	budget	best fixed	P (all)	all	some sol.
MIPLIB 2017	80	80	60 s	H1	0.337	41%	41%
ML4CO item placement	100	100	30 s	H1	0.207	16%	16%
ML4CO load balancing	100	100	30 s	S0	0.067	15%	15%
PGLib-UC	56	33	60 s	C0	0.788	5%	11%
Pooled	336	313	30 s	H1	0.327	19%	25%

Table 1: Headroom across HiGHS, SCIP and cuOpt (six strategies, Table 9): normalised primal integral P of the best fixed strategy and the improvement a per-instance oracle would achieve, one seed. *All*: every attempted instance, with $P = 1$ for every strategy where none finds a solution (23 PGLib-UC cases, all from the FERC system). *Some sol.*: instances where at least one strategy finds a solution; the selectors of Section 4 are trained and evaluated on this cohort. On a five-seed MIPLIB subset about 80 % of the single-seed headroom repeats (Appendix E).

4 Predicting the right configuration

The most direct way to recover the headroom is to predict the right configuration from the instance before solving it. We trained three selectors on 62 features of the instance’s constraint matrix: LightGBM, a small neural network (MLP) and Laya [14], a 421M-parameter decision model (a ModernBERT-large encoder [54] with a classification head) that reads the features as text. Laya was used both untrained and fully fine-tuned, with training targets that give each configuration a weight falling with its cost, $P(a) \propto \exp(-\beta \text{cost}_a)$. Table 2 reports the share of the headroom each selector recovers: 0 means no better than the best single configuration, 1 means as good as the oracle.

At 10 s, fine-tuned Laya recovers 31 % of the headroom (95 % interval 14–48 %, the only interval that excludes zero) and LightGBM 24 %. Compared directly on the same test instances, no superiority was established: the difference is +0.07 of the headroom (interval -0.13 to $+0.34$, paired Wilcoxon $p = 0.67$) at 10 s and +0.06 ($p = 0.09$) at 30 s. On a family left out of training the difference is clear: at 30 s Laya stays level with the best fixed configuration while LightGBM and the MLP fall about 0.4 below it, and Laya beats LightGBM by 0.37 of the headroom (interval 0.16 to 0.68, $p < 10^{-4}$). These intervals hold the fitted models fixed; they do not cover retraining or other folds. The pooled result comes mostly from MIPLIB. Averaged per family instead, every selector falls below each family’s best fixed configuration at 10 s, mainly because of unit commitment, where the headroom is small and a wrong choice costs more than a right one gains; at 30 s only Laya stays above it (Appendix E). Laya answers in 30–56 ms (median, two measurements) and, called every few seconds, does not measurably slow cuOpt on the shared GPU (Appendix F). Computing the features takes 0.3 s (median; at most 5.2 s); this is not charged to the budget and would matter at 10 s, so the 10 s results assume the features are already computed. With these features and models, prediction before the solve recovers part of the headroom and leaves the rest.

5 Choosing versus switching

A controller could also change settings during the solve. Inside SCIP we compared six static settings (default, separation off, pseudo-cost branching, inference branching, heuristics off, aggressive heuristics) with six schedules that switch between them or restart the search, either at 25 % of the budget or after 20 % of the budget without progress. All twelve ran on 45 MIPLIB instances that

policy	seen families (pooled)		unseen family (LOFO)	
	10 s	30 s	10 s	30 s
Best fixed, chosen on training data	+0.00	+0.00	-1.09	+0.00
LightGBM	+0.24	+0.03	-0.03	-0.40
MLP	+0.07	-0.00	-0.08	-0.42
Laya, zero-shot	-0.37	-0.57	-1.04	-1.38
Laya, fine-tuned	+0.31	+0.10	+0.09	-0.02
Oracle	+1.00	+1.00	+1.00	+1.00

Table 2: Selecting a strategy before the solve from 62 static instance features: closed gap between the hindsight single best strategy (0) and the oracle (1); one model for all families, 163 instances with grouped cross-validation (seen families) and 313 with leave-one-family-out (unseen family), all from the cohort where some strategy finds a solution (one FERC unit-commitment case). The hindsight single best defines 0; the best fixed strategy chosen on the training folds can score below it. Intervals resample test instances with the fitted models held fixed; they do not cover retraining or other folds. Positive closed gaps whose 95 % interval excludes zero: the fine-tuned Laya, seen families, 10 s ([0.14, 0.48]); intervals, per-family results and time-to-target are in Appendix E.

SCIP does not solve quickly, with two seeds (Table 3). Each choice is made on one seed and scored on the other, in both directions, averaged. Let B be the mean cost of the single static setting that is best on average, S the mean cost when each instance gets its own best static setting, A the same with all twelve arms (schedules included) available, and R the mean cost when each run gets the arm that won that very run (R is hindsight, not chosen on the other seed). We report

- *choosing*: $(B - S)/B$, the gain from the best static setting per instance;
- *switching*: $(S - A)/B$, what the schedules add on top of that choice;
- *gap to the per-run hindsight oracle*: $(A - R)/B$, what knowing each run’s winner in advance would add. It includes seed-to-seed noise, the limits of choosing on one seed, and anything the tested choices do not use.

Figure 2 shows the result. At 120 s, choosing is worth 11.8 % on P and 6.5 % on PDI, and the six schedules add 2.1 % and 1.6 % (upper 95 % bounds 7.3 % and 3.8 %). At 300 s, on the 20 instances with the most headroom, choosing is worth 20.4 % on P and 11.3 % on PDI, while the schedules add -0.2 % on P and 1.5 % on PDI (interval -0.2 to 4.3 %). With these settings, schedules, instances and budgets, a controller gains by choosing well at the start; the six schedules add little that transfers to a new seed. Other schedules, other actions or budgets beyond 300 s may still gain from switching: DASH [15] gains by switching the branching rule at individual nodes, scored by time to optimality, and Balans [10] by switching large-neighbourhood operators.

The gap to the per-run hindsight oracle is large. With five seeds, the setting that wins on one seed is also the winner averaged over seeds in only 71 % of runs for P (69 % for PDI). The tested predictors do not capture this part; running several copies at once does (Section 7).

6 Agents in the loop

The end-to-end experiment gives an agent a problem, SCIP, a 120 s wall-clock budget and one Cortex-X925 core with 8–9 GiB of memory. SCIP starts with its default settings and calls the agent at solver events (a new best solution or a solved node): first at the first such event after

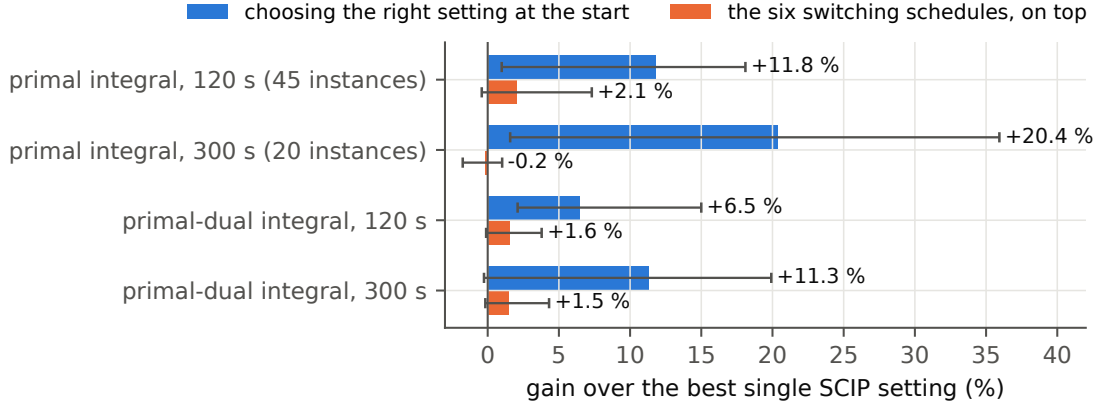


Figure 2: Where the gain inside SCIP comes from: choosing the right static setting per instance (blue) versus switching settings during the solve on top of that choice (orange), both relative to the best single SCIP setting and measured on a held-out seed. 120s: 45 MIPLIB instances; 300s: the 20 with the most headroom.

	static choice	switching, on top	run-level (hindsight)	same winner	best static
primal integral $P(120)$	+11.8% [+1.0, +18.1]	+2.1% [-0.4, +7.3]	5.0 pts	56%	INF
primal-dual integral $PDI(120)$	+6.5% [+2.1, +15.0]	+1.6% [-0.1, +3.8]	3.7 pts	47%	PSC

Table 3: Choosing versus switching inside SCIP: SCIP 10 with six static settings (default, separation off, pseudo-cost branching, inference branching, heuristics off, aggressive heuristics) and six mid-solve schedules (restart at 25 % or after a 20 % stall, branching switches, heuristic bursts), 45 MIPLIB instances, 120s, two seeds, one run per Cortex-X925 core. Gains are over the best single static setting; choices made on one seed and scored on the other; 95 % bootstrap intervals. Run-level: additional gain of an oracle that sees the run itself.

the start, then at the first event after each decision interval. At each call the agent may keep the current setting, change it, or restart the search. SCIP’s time limit is wall-clock time, so the agent’s thinking time is taken from the solver’s budget (Appendix H). Instance features are computed before the start and not charged (median 0.3s, at most 5.2s).

On held-out ML4CO test instances, every agent whose first move is sensible recovers much of the headroom (Table 4). An online bandit with no training (UCB1 over the six settings) improves on the defaults by 21–27 %, and on item placement by about 30 %, better in 28 of 30 runs. The best fixed setting learned from 60 training instances (aggressive heuristics, one setting for both families) gives +29 %; LightGBM predicting a setting per instance +23 %; a bandit that starts from that prediction +29 %; and Laya, fine-tuned for the same first choice, +28 %: it learns to pick aggressive heuristics, the same choice as the best fixed setting.

Language-model agents. We gave the same task to two language models: Ormith-1.5-35B-A3B run locally (NVFP4 on vLLM, thinking off), called every 20s, and OpenAI GPT-6 Luna through a ChatGPT Plus subscription (reasoning effort low), called every 30s. Each call shows the model the same 62 features that Laya and LightGBM see, SCIP’s current state (elapsed time, gap, recent solutions, node rate, current setting, the model’s own earlier decisions) and the six settings with a one-line description each, and asks for one action.

instances $\times$ seeds	default	agents		fixed from $t = 0$		2 copies (2 cores)
		bandit	rules	aggr. hur.	sep. off	
ML4CO, set 1 (20×2)	0.156	0.122	0.160	–	–	0.118
MIPLIB (15×2)	0.390	0.383	0.401	–	–	0.374
ML4CO set 2, item placement (15×2)	0.356	0.250	–	0.245	0.304	–
ML4CO set 2, load balancing (15×2)	0.036	0.038	–	0.035	0.041	–

Table 4: Agent in the loop: primal integral $P(120)$ (lower is better) of agents that start from SCIP’s defaults in a fixed slice (one Cortex-X925 core, 9 GiB, 120 s wall clock including every agent decision). ML4CO instances are held out, from the validation split. Set 1: bandit deciding every 10 s; set 2: every 5 s. Significance tests in Appendix H.

When the agent acts. A fixed setting is applied before SCIP starts, so it also governs presolve. An agent can act only when SCIP first calls it: presolve takes about 0.25 s on these instances, and the first call comes at a median 0.35 s (at most 1.1 s), while SCIP is solving the root node. Part of an agent’s advantage could therefore come from *when* it acts rather than *what* it chooses. Table 5 separates the two. In one session, on 30 held-out ML4CO instances with two seeds, it runs the defaults, aggressive heuristics set before presolve (the best fixed setting on the training instances, one setting for both families), the same setting applied by a fixed agent at the first call, the local language-model agent, and that agent’s first decision alone. Against the defaults these arms gain 33.3 %, 40.6 %, 34.3 % and 31.8 %.

- *Timing.* Applying aggressive heuristics at the first call instead of before presolve is better on 24 of 30 instances ($p = 0.005$). The median per-instance gain is 12.7 % and the mean gain 10.9 %; a few instances lose heavily, so the 95 % bootstrap interval of the mean (−3.9 to +22.3 %) includes zero. The consistent part is on load balancing: better on 15 of 15 instances ($p < 10^{-4}$), although P only moves from 0.037 to 0.032; there the setting applied before presolve scores the same as the defaults. On item placement, 9 instances are better and 6 worse ($p = 0.25$).
- *Language-model agent versus the callback-fixed control.* Against the fixed setting applied at the same callback, the agent is worse on 22 of 30 instances ($p = 0.004$): 10.6 % in the mean (interval −18.9 to −4.0 %) and 3.4 % at the median, mostly on item placement (worse on 12 of 15 instances, $p = 0.03$). Its first move is aggressive heuristics in 49 of 60 runs and “keep the defaults” in the rest. This is an end-to-end comparison: it includes the agent’s inference time (1.3 % of the budget) and its later actions, not only the quality of its choices.
- *Steering.* No benefit was detected from its later decisions: +3.6 % over its first decision alone, with an interval (−10.2 to +15.7 %, $p = 0.67$) that allows a sizeable gain or loss.
- Against the fixed setting applied before presolve, the agent is not significantly different (+1.5 %, $p = 0.31$).

On these two families the language model makes a good first choice without training. It matches the fixed setting applied before presolve, and the same fixed setting applied at the agent’s own first callback beats it.

Why the timing matters. A follow-up session on the same 30 instances (one seed; Table 23 in Appendix H) applied aggressive heuristics before presolve, at the first call, and at the first call

arm	$P(120)$	vs defaults, 95 % interval
SCIP defaults	0.179	–
aggressive heuristics, set before presolve	0.119	+33.3 % [+24.6, +41.4]
aggressive heuristics, set at the first solver event	0.106	+40.6 % [+31.6, +48.0]
LLM agent, first decision only	0.122	+31.8 % [+22.5, +41.2]
LLM agent, every 20 s	0.117	+34.3 % [+24.8, +42.2]
<i>contrast</i> (first vs second arm)	mean (median)	95 % interval, p
timing: heuristics at first event vs heuristics before presolve	+10.9 % (+12.7)	[−3.9, +22.3], $p = 0.0047$
judgement: LLM every 20 s vs heuristics at first event	−10.6 % (−3.4)	[−18.9, −4.0], $p = 0.004$
steering: LLM every 20 s vs LLM first decision	+3.6 % (−0.2)	[−10.2, +15.7], $p = 0.67$
first move: LLM first decision vs heuristics at first event	−14.8 % (−2.7)	[−29.2, −3.6], $p < 10^{-6}$
replication: LLM every 20 s vs heuristics before presolve	+1.5 % (+5.6)	[−12.8, +13.2], $p = 0.31$

Table 5: Timing-matched controls, the primary agent comparison: 30 held-out ML4CO instances (validation split), two seeds, 120 s, all arms in one session. Gains in primal integral, positive when the first arm is better: the gain in the mean, and in parentheses the median per-instance gain. Intervals: instance-level bootstrap of the mean gain; p : Wilcoxon signed-rank over instances. “Heuristics”: the aggressive-heuristics setting; “first event”: the first eligible solver event, where the agents first act.

after 10 s, and recorded SCIP’s statistics. On load balancing SCIP stays at the root node for the whole budget in every arm, spends about 96 of the 120 s in primal heuristics (almost all in RENS) and finds the same solutions in every arm. The arms differ only at the start. Set before presolve, the setting also runs SCIP’s zero-objective heuristic at the start of the solve (0.59 s on average), which delays the first solution from 0.73 s to 1.25 s. Until the first solution the gap counts as 1, so this delay alone costs about $0.5/120 \approx 0.004$ in P , the whole difference between the arms (0.035 against 0.031). Applied at the first call the setting again wins on 15 of 15 instances, and applied after 10 s on 14 of 15. On item placement, where the first solution comes within 0.02 s, the arms do not differ significantly (11 better, 4 worse, $p = 0.36$). The timing effect is therefore specific: one early heuristic that the setting adds only when it is set before the solve, on instances whose first solution takes about a second. It does not show that acting after presolve is better in general.

Other agents (first session). Table 6 comes from an earlier session on the same 30 instances and adds arms that Table 5 lacks. Laya as the agent, fine-tuned for the first choice, improves on the defaults by 37.0 % and uses 0.1 % of the budget. Adding Laya’s probabilities to the local model’s prompt did not improve it (+34.6 %). GPT-6 Luna improved on the defaults by 25.3 % (one seed); its decisions took a median 3.3 s each, 10 % of the budget. In that session the local model scored +37.5 % and the fixed setting before presolve +31.7 %; Table 5 replicates both at +34.3 % and +33.3 %, so arms are compared only within one table. On the primal-dual integral all agents in Table 6 are within 3 % of each other, and their differences come from the 15 item-placement instances: on load balancing every arm of that table scores 0.035–0.038.

arm	runs	$P(120)$	vs default	vs best fixed	thinking
SCIP defaults (no agent)	60	0.184	–	-46.3 %*	–
best fixed setting from training	60	0.126	+31.7 %*	–	–
Laya as the agent (fine-tuned, $t = 0$)	60	0.116	+37.0 %*	+7.9 %	0.1 %
LLM agent: Ornith-1.5-35B-A3B	60	0.115	+37.5 %*	+8.5 %	1.3 %
LLM agent + Laya as a tool	60	0.121	+34.6 %*	+4.3 %	1.5 %
LLM agent: GPT-6 Luna (hosted)	30	0.137	+25.3 %*	-15.3 %	10.1 %

Table 6: The end-to-end agent experiment with LLM agents: 30 held-out ML4CO instances (validation split), SCIP starting from its defaults, one Cortex-X925 core and 8 GiB, 120 s of wall clock that includes every model call; two seeds (the hosted model one). Ornith-1.5-35B-A3B runs locally (NVFP4 on vLLM, thinking off) and is asked every 20 s; GPT-6 Luna (ChatGPT Plus, reasoning effort low) every 30 s. Gains in primal integral; * paired Wilcoxon $p < 0.05$. *Thinking*: share of the budget spent waiting for the agent.

7 Two solvers at once

Run-to-run variation can be used without any prediction by running solvers side by side and keeping the better solution at each moment [20, 40]. On a machine with a GPU the natural pair is a CPU solver and a GPU solver. Estimated from separately recorded runs, HiGHS on one thread plus cuOpt would capture 80 % of the headroom between solvers on the instances where some configuration finds a solution (Table 7). Run live on the 80 MIPLIB instances at 60 s, launched together, with both arms of each comparison interleaved in one session in a fixed order per instance, and the clock started at launch:

- HiGHS on one core with cuOpt on the GPU improves on HiGHS alone by 30.4 % (95 % interval 17.7 to 44.2 %; better on 61 instances, worse on 13, $p < 10^{-8}$);
- SCIP with cuOpt improves on SCIP alone by 21.5 % (interval 9.3 to 36.9 %; better on 28, worse on 13, $p < 10^{-4}$) on the first 44 instances in job-file order; this run stopped at the end of its scheduled window.

Scores use each solver’s best solution; cuOpt’s are feasible at its own tolerance of 10^{-4} , and two of the 140 checked exceed 10^{-6} (Appendix D). Within SCIP, two copies with different seeds improve P by 7 % and PDI by 5.5 %. Each of these uses more compute than a single run: cuOpt adds the GPU and five host cores.

The two solvers can also exchange solutions. When cuOpt’s solutions are passed to SCIP through SCIP’s heuristic interface, SCIP’s own primal integral improves by 14 % ($p < 10^{-3}$; better on 47 instances, worse on 24). This matters when one solver’s result is needed together with its bound. The pair’s best-of value does not improve beyond the plain race, because cuOpt already held those solutions. Passing SCIP’s solutions to cuOpt as well costs 15 %: cuOpt switches off its presolve once it accepts outside solutions. Appendix G describes the interfaces, including one that crashes SCIP.

8 Discussion

What this means in practice. For a user with a time budget, within the families, hardware and budgets (30–300 s) we tested:

setting	portfolio	P gain	PDI gain
HiGHS/SCIP/cuOpt study, pooled (313 inst.)	HiGHS (1 thread) cuOpt (GPU)	22 %	–
SCIP, 20 MIPLIB inst., 120 s	2 copies of the best setting	7%	5%
SCIP, 20 MIPLIB inst., 120 s	3 copies of the best setting	9%	8%
SCIP, 20 MIPLIB inst., 120 s	5 copies of the best setting	11%	9%

Table 7: Parallel portfolios, computed exactly from recorded trajectories: best incumbent (and bound) of runs executed side by side, against the best single strategy. HiGHS||cuOpt captures 80 % of the cross-solver oracle headroom; SCIP copies differ only in the random seed.

1. Run a GPU solver beside the CPU solver: +30 % over HiGHS alone and +21 % over SCIP alone (44 instances), with no model.
2. Choose the setting once, at the start. On these ML4CO families one fixed setting chosen on training instances, and a fine-tuned Laya, improve on SCIP’s defaults by about a third; an untrained local language model finds the same first move but does not beat the fixed setting applied at the same callback. Where the first solution takes about a second, check whether a setting adds work before it: on load balancing, aggressive heuristics set before the solve delayed the first solution by half a second.
3. If one solver’s result is needed, pass the other solver’s solutions in through the supported heuristic interface (+14 %).

For learned controllers such as Laya, the useful task is therefore to diagnose the instance early, measured against the best fixed setting and an online bandit, rather than to steer the solver continuously.

Future research. (i) A new test cohort from families where no single setting is best on the training data, with the protocol and one primary comparison fixed in advance and the timing-matched controls as baselines. (ii) A short diagnostic run, charged to the budget, before a single choice, with a rule to keep the default when the predicted gain is small; and richer state (root LP statistics, early solutions) for learned selectors. (iii) Budgets longer than 300 s, with a small set of schedules fixed in advance. (iv) Language-model agents with an explicit thinking-time budget that decide *when* to act, not only what to do. (v) Which solvers to run or pair under a compute budget, with CPU, GPU, memory and energy counted, and exchange in both directions with HiGHS as the CPU partner. (vi) Using the GPU *inside* a MIP solver: GPU LP solvers of the PDLP type for relaxations in a CPU branch-and-bound, and GPU heuristic populations. These are solver-engineering projects, and several GPU jobs at once need scheduling: two slowed each other in our measurements (Appendix F).

Limitations. All timings come from one DGX Spark. Whether the conclusions transfer to other hardware is untested: relative results can change with the balance of CPU and GPU and with memory contention. SCIP is the only solver studied in depth, the action sets are small and hand-picked, most results use two seeds, the language-model agents saw two instance families, and budgets are at most 300 s. Tests are paired over instances (seeds averaged). Apart from the 10 % headroom threshold set in advance, the comparisons are exploratory rather than pre-registered, including the principal ones named in Section 2. An interval that includes zero is not evidence that two arms are equal, so we report its bounds. Only final solutions are checked independently. Instance features are computed before the clock starts (Appendix J).

9 Conclusion

In HiGHS, SCIP and cuOpt, choosing settings per instance could lower the primal integral by 19 % over all attempted instances and 25 % where some configuration finds a solution (one seed; about 80 % of it repeats on a five-seed subset), and inside SCIP most of it comes from the initial choice rather than from the switching schedules we tried. Running cuOpt beside HiGHS recovers 30 % and beside SCIP 21 % (44 instances) with no model. One fixed setting chosen on training instances improves on SCIP’s defaults by a third on the ML4CO families; an untrained local language model finds the same first move, but the fixed setting applied at the same callback beats it on 22 of 30 instances. Two questions remain open: whether an agent can beat the right fixed choice on families where that choice is not obvious, and whether switching helps at budgets beyond 300 s. The library, the experiment definitions, the archived trajectories and `optopt reproduce` are released so that both can be tested against the same baselines.

References

- [1] Tobias Achterberg and Timo Berthold. Hybrid branching. In *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems (CPAIOR 2009)*, Lecture Notes in Computer Science, pages 309–311. Springer, 2009. doi: 10.1007/978-3-642-01929-6_23.
- [2] Daniel Anderson, Gregor Hendel, Pierre Le Bodic, and Merlin Viernickel. Clairvoyant restarts in branch-and-bound search using online tree-size estimation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 1427–1434, 2019. doi: 10.1609/aaai.v33i01.33011427.
- [3] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021. doi: 10.1016/j.ejor.2020.07.063. Preprint: <https://arxiv.org/abs/1811.06128>.
- [4] Timo Berthold. Measuring the impact of primal heuristics. *Operations Research Letters*, 41(6):611–614, 2013. doi: 10.1016/j.orl.2013.08.007. Open version: <https://opus4.kobv.de/opus4-zib/frontdoor/index/index/docId/1788> (ZIB-Report 13-17).
- [5] Timo Berthold, Gregor Hendel, and Thorsten Koch. From feasibility to improvement to proof: Three phases of solving mixed-integer programs. *Optimization Methods and Software*, 33(3): 499–517, 2018. doi: 10.1080/10556788.2017.1392519. Preprint: ZIB-Report 16-78, <https://opus4.kobv.de/opus4-zib/files/6160/zib-paper.pdf>.
- [6] Mauro Birattari, Thomas Stützle, Luís Paquete, and Klaus Varrentrapp. A racing algorithm for configuring metaheuristics. In W. B. Langdon et al., editors, *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2002)*, pages 11–18. Morgan Kaufmann, 2002. Open version: <https://eden.dei.uc.pt/~paquete/papers/gecco2002.pdf>.
- [7] Bernd Bischl, Pascal Kerschke, Lars Kotthoff, Marius Lindauer, Yuri Malitsky, Alexandre Fréchette, Holger H. Hoos, Frank Hutter, Kevin Leyton-Brown, Kevin Tierney, and Joaquin Vanschoren. ASlib: A benchmark library for algorithm selection. *Artificial Intelligence*, 237:41–58, 2016. doi: 10.1016/j.artint.2016.04.003. Preprint: <https://arxiv.org/abs/1506.02465>.

- [8] Pierre Bonami, Andrea Lodi, and Giulia Zarpellon. Learning a classification of mixed-integer quadratic programming problems. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2018)*, volume 10848 of *Lecture Notes in Computer Science*, pages 595–604. Springer, 2018. doi: 10.1007/978-3-319-93031-2_43.
- [9] Junyang Cai, Taoan Huang, and Bistra Dilkina. Multi-task representation learning for mixed integer linear programming. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2025)*, *Lecture Notes in Computer Science*, pages 134–151. Springer, 2025. doi: 10.1007/978-3-031-95973-8_9. Preprint: <https://arxiv.org/abs/2412.14409>.
- [10] Junyang Cai, Serdar Kadioglu, and Bistra Dilkina. Balans: Multi-armed bandits-based adaptive large neighborhood search for mixed-integer programming problems. In *Proceedings of the 34th International Joint Conference on Artificial Intelligence (IJCAI 2025)*, pages 2566–2574, 2025. doi: 10.24963/ijcai.2025/286.
- [11] Rodolfo Carvajal, Shabbir Ahmed, George L. Nemhauser, Kevin C. Furman, Vikas Goel, and Yufen Shao. Using diversification, communication and parallelism to solve mixed-integer linear programs. *Operations Research Letters*, 42(2):186–189, 2014. doi: 10.1016/j.orl.2013.12.012.
- [12] Antonia Chmiela, Elias B. Khalil, Ambros Gleixner, Andrea Lodi, and Sebastian Pokutta. Learning to schedule heuristics in branch-and-bound. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021. Preprint: <https://arxiv.org/abs/2103.10294>.
- [13] Antonia Chmiela, Ambros Gleixner, Pawel Lichocki, and Sebastian Pokutta. Online learning for scheduling MIP heuristics. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2023)*, *Lecture Notes in Computer Science*, pages 114–123. Springer, 2023. doi: 10.1007/978-3-031-33271-5_8. Preprint: <https://arxiv.org/abs/2304.03755>.
- [14] Convai Innovations. Laya: Multilingual, non-autoregressive system 1 decision model (model card). <https://huggingface.co/convaiinnovations/laya>, 2026. Hugging Face model repository, created 2026-09-18, last modified 2026-09-20; Apache-2.0. Code: <https://github.com/NandhaKishorM/laya>. No paper; accessed 2026-09-23.
- [15] Giovanni Di Liberto, Serdar Kadioglu, Kevin Leo, and Yuri Malitsky. DASH: Dynamic approach for switching heuristics. *European Journal of Operational Research*, 248(3):943–953, 2016. doi: 10.1016/j.ejor.2015.08.018. Preprint: <https://arxiv.org/abs/1307.4689>.
- [16] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. Preprint: <https://arxiv.org/abs/1906.01629>.
- [17] Maxime Gasse, Quentin Cappart, et al. The machine learning for combinatorial optimization competition (ML4CO): Results and insights. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, volume 176 of *Proceedings of Machine Learning Research*, pages 220–231, 2022. Preprint: <https://arxiv.org/abs/2203.02433>.
- [18] Alexander Georges, Ambros Gleixner, Gorana Gojic, Robert Lion Gottwald, David Haley, Gregor Hendel, and Bartłomiej Matejczyk. Feature-based algorithm selection for mixed integer

- programming. ZIB-Report 18-17, Zuse Institute Berlin, 2018. URL <https://opus4.kobv.de/opus4-zib/frontdoor/index/index/docId/6836>.
- [19] Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, Marco Lübbecke, Hans D. Mittelmann, Derya Ozyurt, Ted K. Ralphs, Domenico Salvagnin, and Yuji Shinano. MIPLIB 2017: Data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation*, 13(3):443–490, 2021. doi: 10.1007/s12532-020-00194-3. Open version: <https://link.springer.com/article/10.1007/s12532-020-00194-3> (open access, CC BY 4.0).
 - [20] Carla P. Gomes and Bart Selman. Algorithm portfolios. *Artificial Intelligence*, 126(1–2):43–62, 2001. doi: 10.1016/S0004-3702(00)00081-3. Open version: <https://www.sciencedirect.com/science/article/pii/S0004370200000813> (Elsevier open archive).
 - [21] He He, Hal Daumé III, and Jason Eisner. Learning to search in branch and bound algorithms. In *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, 2014. URL https://proceedings.neurips.cc/paper_files/paper/2014/hash/533d190f5aa2926b2a8a30c8bea0e05d-Abstract.html.
 - [22] Gregor Hendel. Exploiting solving phases for mixed-integer programs. In *Operations Research Proceedings 2015*, pages 3–9. Springer, 2017. doi: 10.1007/978-3-319-42902-1_1. Preprint: https://opus4.kobv.de/opus4-zib/files/5709/Hendel_ExploitingSolvingPhases.pdf.
 - [23] Gregor Hendel, Matthias Miltenberger, and Jakob Witzig. Adaptive algorithmic behavior for solving mixed integer programs using bandit algorithms. In Bernard Fortz and Martine Labbé, editors, *Operations Research Proceedings 2018*, pages 513–519. Springer, 2019. doi: 10.1007/978-3-030-18500-8_64. Open version: <https://opus4.kobv.de/opus4-zib/frontdoor/index/index/docId/6956> (ZIB-Report 18-36).
 - [24] Taoan Huang, Aaron Ferber, Yuandong Tian, Bistra Dilikina, and Benoit Steiner. Searching large neighborhoods for integer linear programs with contrastive learning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 13869–13890, 2023. URL <https://proceedings.mlr.press/v202/huang23g.html>. Preprint: <https://arxiv.org/abs/2302.01578>.
 - [25] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Automated configuration of mixed integer programming solvers. In *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems (CPAIOR 2010)*, Lecture Notes in Computer Science, pages 186–202. Springer, 2010. doi: 10.1007/978-3-642-13520-0_23. Open version: <https://www.cs.ubc.ca/~hutter/papers/10-CPAIOR-MIP-Config.pdf>.
 - [26] IEEE PES Task Force on Benchmarks for Validation of Emerging Power System Algorithms. PGLib-UC: Benchmarks for the unit commitment problem. <https://github.com/power-grid-lib/pglib-uc>, 2026. JSON case files (ca, ferc, rts_gmlc), CC BY licence.
 - [27] Serdar Kadioglu, Yuri Malitsky, Meinolf Sellmann, and Kevin Tierney. ISAC – instance-specific algorithm configuration. In *Proceedings of the 19th European Conference on Artificial Intelligence (ECAI 2010)*, Frontiers in Artificial Intelligence and Applications. IOS Press, 2010. doi: 10.3233/978-1-60750-606-5-751.

- [28] Serdar Kadioglu, Yuri Malitsky, Ashish Sabharwal, Horst Samulowitz, and Meinolf Sellmann. Algorithm selection and scheduling. In *Principles and Practice of Constraint Programming (CP 2011)*, volume 6876 of *Lecture Notes in Computer Science*, pages 454–469. Springer, 2011. doi: 10.1007/978-3-642-23786-7_35.
- [29] Elias B. Khalil, Pierre Le Bodic, Le Song, George Nemhauser, and Bistra Dilkina. Learning to branch in mixed integer programming. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016. doi: 10.1609/aaai.v30i1.10080.
- [30] Elias B. Khalil, Bistra Dilkina, George L. Nemhauser, Shabbir Ahmed, and Yufen Shao. Learning to run heuristics in tree search. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 659–666, 2017. doi: 10.24963/ijcai.2017/92.
- [31] Bernard Knueven, James Ostrowski, and Jean-Paul Watson. On mixed-integer programming formulations for the unit commitment problem. *INFORMS Journal on Computing*, 2020. doi: 10.1287/ijoc.2019.0944. Open version: <https://www.osti.gov/biblio/1670724> (accepted manuscript, SAND2019-13440J).
- [32] Markus Kruber, Marco E. Lübbecke, and Axel Parmentier. Learning when to use a decomposition. In *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2017)*, *Lecture Notes in Computer Science*, pages 202–210. Springer, 2017. doi: 10.1007/978-3-319-59776-8_16.
- [33] Abdel Ghani Labassi, Didier Chételat, and Andrea Lodi. Learning to compare nodes in branch and bound with graph neural networks. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/cf5bb18807a3e9cfaaa51e667e18f807-Abstract-Conference.html. Preprint: <https://arxiv.org/abs/2210.16934>.
- [34] Connor Lawless, Yingxi Li, Anders Wikum, Madeleine Udell, and Ellen Vitercik. LLMs for cold-start cutting plane separator configuration. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2025)*, *Lecture Notes in Computer Science*, pages 51–69. Springer, 2025. doi: 10.1007/978-3-031-95976-9_4.
- [35] Hongpei Li, Ziyang He, Yufei Wang, Wenting Tu, Shanwen Pu, Qi Deng, and Dongdong Ge. BenLOC: A benchmark for learning to configure MIP optimizers, 2025. Preprint: <https://arxiv.org/abs/2506.02752>.
- [36] Marius Lindauer, Jan N. van Rijn, and Lars Kotthoff. The algorithm selection competitions 2015 and 2017. *Artificial Intelligence*, 272:86–100, 2019. doi: 10.1016/j.artint.2018.10.004. Preprint: <https://arxiv.org/abs/1805.01214>.
- [37] Andrea Lodi and Andrea Tramontani. Performance variability in mixed-integer programming. In *Theory Driven by Influential Applications*, *INFORMS TutORials in Operations Research*, pages 1–12. INFORMS, 2013. doi: 10.1287/educ.2013.0112.
- [38] Andrea Lodi and Giulia Zarpellon. On learning and branching: A survey. *TOP*, 25(2):207–236, 2017. doi: 10.1007/s11750-017-0451-6.
- [39] Yidong Luo, Xuemin Chen, Chenguang Wang, Fangzhou Zhu, Tao Zhong, and Tianshu Yu. GRIMIP: A general framework for instance-specific configuration of MIP solvers using LLMs, 2026.

- [40] Gioni Mexi and Daniel Rehfeldt. Race, exchange, improve: Finding high-quality MIP solutions quickly, 2026.
- [41] Vinod Nair, Sergey Bartunov, Felix Gimeno, Ingrid von Glehn, Pawel Lichocki, Ivan Lobov, Brendan O’Donoghue, Nicolas Sonnerat, et al. Solving mixed integer programs using neural networks, 2020. Preprint: <https://arxiv.org/abs/2012.13349>.
- [42] NVIDIA. NVIDIA cuOpt: GPU-accelerated decision optimization (LP, MILP, routing). <https://github.com/NVIDIA/cuopt>, 2026. Open source (Apache-2.0); release 26.08 used here.
- [43] Max B. Paulus, Giulia Zarpellon, Andreas Krause, Laurent Charlin, and Chris J. Maddison. Learning to cut by looking ahead: Cutting plane selection via imitation learning. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, 2022.
- [44] Alessio Pellegrino, Özgür Akgün, Nguyen Dang, Zeynep Kiziltan, and Ian Miguel. Transformer-based feature learning for algorithm selection in combinatorial optimisation. In *31st International Conference on Principles and Practice of Constraint Programming (CP 2025)*, volume 340 of *LIPIcs*, pages 31:1–31:22, 2025. doi: 10.4230/LIPIcs.CP.2025.31.
- [45] John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages 65–118. Elsevier, 1976. doi: 10.1016/S0065-2458(08)60520-3. Open version: <https://docs.lib.purdue.edu/cstech/99/> (Purdue CS technical report of the same title, 1975).
- [46] Lara Scavuzzo, Karen Aardal, Andrea Lodi, and Neil Yorke-Smith. Machine learning augmented branch and bound for mixed integer linear programming. *Mathematical Programming*, 217(1–2):123–166, 2024. doi: 10.1007/s10107-024-02130-y. Published online 2024; Preprint: <https://arxiv.org/abs/2402.05501>.
- [47] Jialin Song, Ravi Lanka, Yisong Yue, and Bistra Dilkina. A general large neighborhood search framework for solving integer linear programs. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. Preprint: <https://arxiv.org/abs/2004.00422>.
- [48] Nicolas Sonnerat, Pengming Wang, Ira Ktena, Sergey Bartunov, and Vinod Nair. Learning a large neighborhood search algorithm for mixed integer programs, 2021. Preprint: <https://arxiv.org/abs/2107.10201>.
- [49] Stefan Szeider. Algorithm selection with zero domain knowledge via text embeddings, 2026.
- [50] Yunhao Tang, Shipra Agrawal, and Yuri Faenza. Reinforcement learning for integer programming: Learning to cut. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 9367–9376, 2020. URL <https://proceedings.mlr.press/v119/tang20a.html>.
- [51] Romeo Valentin, Claudio Ferrari, Jérémy Scheurer, Andisheh Amrollahi, Chris Wendler, and Max B. Paulus. Instance-wise algorithm configuration with graph neural networks, 2022. Preprint: <https://arxiv.org/abs/2202.04910>.
- [52] Diederick Vermetten, Hao Wang, Thomas Bäck, and Carola Doerr. Towards dynamic algorithm selection for numerical black-box optimization: Investigating BBOB as a use case. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2020)*, pages 654–662, 2020. doi: 10.1145/3377930.3390189. Preprint: <https://arxiv.org/abs/2006.06586>.

- [53] Zhihai Wang, Xijun Li, Jie Wang, Yufei Kuang, Mingxuan Yuan, Jia Zeng, Yongdong Zhang, and Feng Wu. Learning cut selection for mixed-integer linear programming via hierarchical sequence model. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=Zob4P9bRNcK>.
- [54] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, et al. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*, pages 2526–2547, 2025. doi: 10.18653/v1/2025.acl-long.127.
- [55] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008. doi: 10.1613/jair.2490.
- [56] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Hydra-MIP: Automated algorithm configuration and selection for mixed integer programming. In *RCRA Workshop on Experimental Evaluation of Algorithms for Solving Problems with Combinatorial Explosion at IJCAI 2011*, 2011. URL <https://www.cs.ubc.ca/~hoos/Publ/XuEtAl11.pdf>.
- [57] Ali Emre Yilmaz, Junyang Cai, Serdar Kadioglu, and Bistra Dilkina. ParBalans: Parallel multi-armed bandits-based adaptive large neighborhood search, 2025.

Naming. The appendix and the repository call the cross-solver study (HiGHS, SCIP, cuOpt; Sections 3, 4 and 7) *Track A*, and the SCIP studies (Sections 5 and 6) *Track B*, with B0–B3 its experiments.

A Related work in detail, and work behind paywalls

We searched for prior work that uses a learned, neural or foundation model for the solver-side functions studied here (per-instance configuration, switching during the solve, selection across solvers), in four sweeps: branching and node selection; primal heuristics and large-neighbourhood search; configuration, selection and dynamic control; and LLMs, foundation models and GPU solvers (159 works), followed by a re-check of the novelty claims (21 more; the notes, with the evidence for each entry, are in the repository vault under `40-literature/prior-art`). Learned per-instance configuration of one solver is well established [35, 51, 56]; BenLOC’s finding that learned gains over the per-dataset best stay under 5% and that tree models beat deep ones agrees with our Table 2. The closest foundation-model work, GRIMIP [39], reports primal-dual integrals on the same ML4CO families and MIPLIB, but tunes Gurobi with about ten solver runs per instance; our selectors make one prediction inside the budget. Choosing among SCIP settings per instance from features was studied by Georges et al. [18], and choosing among MIP solvers (CBC, CPLEX, Gurobi, SCIP, Xpress) is the ASlib MIP-2016 scenario of the 2017 selection competition, where the best selector closed about half of the gap to the oracle [36]. What we did not find — a GPU solver in the portfolio, the primal integral as the selection target, a fine-tuned pretrained encoder as the selector — rests on negative searches in which part of the 2025–2026 preprint literature could only be reached by web search, so we make no priority claim. Relevant works we could not read legally are listed separately in Appendix A.1.

A.1 Paywalled research

This subsection is the only place in the paper that cites work we could not read. The following works appear relevant but are behind publisher paywalls, and we found no legal free copy (preprint, accepted manuscript, repository or thesis version). *We have not read them in full.* They were found through accessible papers that cite or describe them, or judged relevant from their abstract and metadata; the descriptions below are limited to what those sources state. Per-instance configuration by clustering (ISAC, 27) and algorithm selection combined with a schedule of short runs (3S, 28); running differently configured copies of one MILP solver in parallel with shared information [11]; learned per-instance decisions inside a solver, whether to use a decomposition [32] or whether CPLEX should linearise a quadratic problem [8]; SCIP’s non-learned combination of branching scores [1]; a survey of learning for branching [38]; and the standard account of performance variability under seeds [37], which underlies the parallel copies of Table 7. Where a work in this paper is paywalled but a legal preprint exists, the bibliography points to the preprint.

B Hardware, software and resources

All experiments ran on one NVIDIA DGX Spark: GB10 Grace–Blackwell superchip (10 Cortex-X925 + 10 Cortex-A725 cores, Blackwell GPU, 128GB coherent unified memory, of which the operating system reports 121 GiB), Ubuntu 24.04, kernel 6.17, driver 580.173.02, CUDA 13.0. Solvers: HiGHS 1.15.1 (`highspy`), SCIP 10.0 (PySCIPOpt 6.2.1), cuOpt 26.08 (`cuopt-cu13`); all installed from released wheels. Laya: the published `convaiinnovations/laya` checkpoint (421.3M parameters), PyTorch 2.14 (CUDA 13.0 build), bf16. Other workloads on the machine (two local LLM servers) were stopped for the whole Track A measurement window. Table 24 lists the resources allocated to every experiment.

Heterogeneous cores. The GB10 has ten Cortex-X925 cores at 3.9 GHz and ten Cortex-A725 cores at 2.8 GHz. Pinned, SCIP explored 4,686 nodes on an X925 core and 2,413 on an A725 core in the same 20 s (tr12-30). Track A and B0 runs were not pinned, so their effective budgets varied by up to about $2\times$ at random (no strategy favoured); all later experiments pin one run per X925 core and use A725 cores only for untimed work.

Memory. From Track B on, every solver run is confined to its own systemd scope with a hard memory cap (8–9 GiB, no swap) and the solver’s own limit set 1 GiB lower; the runner refuses to start if $\text{lanes} \times \text{cap}$ plus a 16 GiB margin exceeds available memory.

C Instances

- **MIPLIB 2017** [19]: 80 of the 240 benchmark-set instances, chosen deterministically for diversity — at most one instance per MIPLIB group, at most 2.5M non-zeros, not infeasible, stratified by MIPLIB status and size tercile. Budget $T = 60$ s. Reference: published optimum or best known value. The set is 96 % “easy” by MIPLIB’s definition (solved within an hour by some solver), which is not the same as easy within 60 s: only 56 of 80 reach a 1 % gap within 60 s under any strategy.
- **ML4CO item placement and load balancing** [17]: two homogeneous generated families from the NeurIPS 2021 competition. Training instances 0–74 of each family’s official training

split (cuOpt) / 0–99 (CPU strategies), and the first 25 instances of the official validation split as test instances. Budget $T = 30$ s. Reference: best value found by any of our runs.

- **PGLib-UC** [26]: all 56 unit-commitment cases (three base systems: CAISO, FERC, RTS-GMLC), built with the benchmark’s own reference Pyomo model [31] (about 250k rows and 250k columns, 40 % binary). Budget $T = 60$ s. Split by base system.

The budgets are shorter than the spec’s 300 s screening: the machine has one GPU, cuOpt is the throughput bottleneck, and one night was available. The 10 s and 30 s views are cut from the same trajectories. On MIPLIB, HiGHS and SCIP defaults were additionally run for 300 s to place instances into the spec’s SHORT/MEDIUM/HARD bands (Appendix).

family (system)	attempted	some strategy finds a solution	scored
MIPLIB 2017	80	75	80
ML4CO item placement	100	100	100
ML4CO load balancing	100	100	100
PGLib-UC, CA	20	20	20
PGLib-UC, FERC	24	1	1
PGLib-UC, RTS-GMLC	12	12	12
all	336	308	313

Table 8: Instances in the cross-solver grid, run with all six strategies (one seed, 30 s view). An instance is scored on the conditional cohort when it has a reference value, which requires some strategy to find a solution; in the full cohort the unsolved instances score $P = 1$ for every strategy. MIPLIB instances always have a published reference, so its unsolved instances are scored in both.

D Metrics and verification

For incumbent $z(t)$ and reference z^* the primal gap is $\gamma(t) = |z(t) - z^*| / \max(|z(t)|, |z^*|)$, with $\gamma = 1$ before the first incumbent or on a sign mismatch [4]; $P(T) = \frac{1}{T} \int_0^T \gamma$. Berthold’s definition assumes z^* is optimal and leaves one case open: an incumbent *better* than the reference. We score it as $\gamma = 0$ when it is within 10^{-4} (relative) of the reference — solver tolerance, 21 of 3,768 Track A runs — and as invalid ($\gamma = 1$) beyond that. The one invalid case was a placeholder objective (0.0 against a proven optimum of 576.34) that cuOpt returned with an error status, outside the budget; it affects no result. Where the reference is a best known value rather than a proven optimum, a better incumbent can also mean the reference is out of date; a reusable policy should check such a solution and, if it is feasible, update the reference rather than score it as invalid. Time-to-target is the first time $\gamma \leq 1\%$, censored at $2T$ and summarised by the shifted geometric mean.

Recording SCIP’s incumbents. At SCIP’s “best solution found” event the solver’s primal bound still holds the *previous* incumbent, so reading it there records incumbent $k - 1$ at the time of incumbent k . Because every new best solution fires its own event, the true sequence is recoverable exactly (the value read at event $k + 1$, and the final primal bound for the last one); every SCIP trajectory in this paper records each incumbent at its own time. Reading the bound at the event would favour settings that reach branch-and-bound nodes sooner, such as separation off. HiGHS and cuOpt callbacks report the new solution’s own objective and were not affected (checked for

HiGHS: at every improving-solution event its reported primal bound equals the new solution’s objective).

Integration and tests. The primal-dual integral is integrated on the exact event times, with regression tests against an independent reference. Paired tests are over instances, with the seeds of an instance averaged. In the agent runs other than the timing-matched controls, an agent called at a new-solution event saw the previous incumbent’s value (the recording lag above, in the agent’s state); the timing-matched control runs give agents the current value.

Solution checking. From Track B on, every SCIP run stores its final incumbent (earlier incumbents, which also enter $P(T)$, are not stored), and an independent checker re-reads the model with HiGHS and verifies bounds, integrality, every row and the objective in exact rational arithmetic at tolerance 10^{-6} (rows by a float pass with an exact re-check of any row near the tolerance). HiGHS and cuOpt solutions from the live pair runs are stored and checked too. Of 4,332 stored solutions (4,125 SCIP, 67 HiGHS, 140 cuOpt), 4,326 are strictly within 10^{-6} . Four SCIP solutions (one instance, the aggressive-heuristics arm) exceed it by about 10^{-13} relative, i.e. they sit at the solver’s own feasibility tolerance. Two cuOpt solutions exceed it by up to $3 \cdot 10^{-6}$ in integrality: cuOpt’s default feasibility tolerances are 10^{-4} , so its incumbents are not guaranteed at the stricter standard used here. Two further stored solutions have no matching trace and are not scored. A deliberately corrupted solution is rejected.

E Track A details: selection across HiGHS, SCIP and cuOpt

id	solver	settings (non-default only)
H0	HiGHS 1.15.1	default
H1	HiGHS 1.15.1	<code>mip_heuristic_effort</code> =0.3 (default 0.05)
S0	SCIP 10.0	default
S1	SCIP 10.0	heuristics emphasis AGGRESSIVE
C0	cuOpt 26.08	default (GPU + 3 CPU threads)
C1	cuOpt 26.08	<code>mip_cut_passes</code> =0 (default 10), RINS time limit 10 s (default 3 s)

Table 9: Track A strategy portfolio, chosen a priori (not tuned).

Budgets were 60 s (MIPLIB, PGLib-UC) and 30 s (ML4CO); 10 s and 30 s views are cut from the same trajectories. HiGHS and SCIP ran single-threaded on twelve concurrent CPU lanes and cuOpt on two GPU lanes.

Selectors. Features: 62 static statistics of the constraint matrix (sizes, variable and row types, set-partitioning and knapsack row shares, coefficient, cost and right-hand-side ranges, degrees), computed in well under a second except on PGLib-UC, where parsing dominates (Table 18). Light-GBM: one cost regressor per strategy. MLP: two hidden layers of 64 units, soft targets with $\beta = 20$. Hand rules: four rules committed before any result was seen. Laya: the published checkpoint asked a six-way choice question over a ~ 260 -token text rendering of the features (zero-shot), and the same checkpoint fully fine-tuned with the same soft targets (8 epochs, AdamW, encoder $2 \cdot 10^{-5}$, head 10^{-4} , one seed per fold, 70–100 s per fold on the GPU). Pooled evaluation: one model for all families, five folds, every instance tested once. LOFO: train on three families, test on the fourth.

T	family	n	H0	H1	S0	S1	C0	C1	SBS	VBS	headroom
10	MIPLIB 2017	80	0.532	0.528	0.551	0.548	0.523	0.548	C0	0.356	32%
10	ML4CO item placement	100	0.495	0.392	0.515	0.448	0.734	0.721	H1	0.344	12%
10	ML4CO load balancing	100	0.256	0.247	0.166	0.250	0.219	0.241	S0	0.139	16%
10	PGLib-UC	33	0.883	0.873	0.993	0.879	0.819	0.820	C0	0.808	1%
10	All (pooled)	313	0.469	0.431	0.463	0.456	0.524	0.534	H1	0.331	23%
30	MIPLIB 2017	80	0.413	0.406	0.438	0.443	0.407	0.440	H1	0.243	40%
30	ML4CO item placement	100	0.261	0.207	0.363	0.282	0.466	0.432	H1	0.174	16%
30	ML4CO load balancing	100	0.095	0.092	0.067	0.095	0.094	0.119	S0	0.057	15%
30	PGLib-UC	33	0.755	0.748	0.802	0.835	0.785	0.758	H1	0.701	6%
30	All (pooled)	313	0.299	0.278	0.334	0.322	0.365	0.368	H1	0.210	25%
60	MIPLIB 2017	80	0.348	0.337	0.367	0.374	0.341	0.380	H1	0.199	41%
60	PGLib-UC	33	0.696	0.685	0.722	0.763	0.640	0.697	C0	0.572	11%
60	All (pooled)	113	0.449	0.439	0.471	0.487	0.428	0.473	C0	0.308	28%

Table 10: Mean normalised primal integral $P(T)$ (lower is better) per strategy, seed 0. Bold: single best strategy (SBS) of the family. VBS: per-instance oracle. Headroom: $(\text{SBS} - \text{VBS})/\text{SBS}$.

strategy	mean s.d. of $P(60)$	median s.d.	max range
H0		0.025	0.011
H1		0.019	0.011
S0		0.020	0.004
S1		0.032	0.005

Table 11: Run-to-run variability across seeds on the 12-instance MIPLIB variance subset (HiGHS and SCIP strategies, seeds 0–4; cuOpt extra seeds were cut for time). Among these four strategies the per-seed winner equals the seed-averaged winner on 65% of (instance, seed) pairs. A single-seed oracle appears 31.8% better than the SBS on its own seed and realises 26.3% on held-out seeds (true seed-mean oracle: 29.6%).

F GPU contention between Laya and cuOpt

Does running Laya slow cuOpt on the shared GPU? On five MIPLIB instances where cuOpt keeps improving between 5 and 30 s, cuOpt C0 ran for 30 s under four conditions, two repetitions each, with condition order rotated (Table 19). Keeping Laya *loaded but idle* (B, about 5 GB of the unified memory) and making *one decision every 5 s* (C, the duty cycle of a dynamic controller) change cuOpt’s primal integral by +0.014 and +0.009 on average — within run-to-run noise (median paired differences −0.004 and 0.000). Running Laya *continuously* (D) doubles cuOpt’s primal integral (0.226→0.452; worse in 10 of 10 paired runs) even though the SM clock stays at 2.39 GHz: the two workloads compete for the GPU and its 273 GB/s memory system, not for power. Laya’s own latency is 30 ms median (50 ms p90) beside cuOpt. The practical rule: **call the decision model sequentially or at a low duty cycle, never concurrently in a loop**. At a 5 s decision interval the raw inference overhead is about 0.6 %.

band	n	H0	H1	S0	S1	C0	C1	VBS	headroom
trivial (<5 s)	10	0.087	0.077	0.130	0.133	0.190	0.284	0.011	86%
SHORT (5–30 s)	11	0.188	0.166	0.159	0.182	0.195	0.198	0.059	63%
MEDIUM (30–120 s)	16	0.221	0.200	0.224	0.289	0.293	0.313	0.061	70%
HARD (120–300 s)	6	0.256	0.249	0.395	0.419	0.463	0.466	0.141	43%
unsolved in 300 s	37	0.535	0.532	0.550	0.525	0.426	0.475	0.360	16%

Table 12: MIPLIB subset by the spec’s difficulty bands (time to optimality of the faster of HiGHS and SCIP defaults in a 300 s run, 12 concurrent CPU runs). Entries: mean $P(60)$ per strategy (bold = best), oracle, headroom over the band’s best fixed strategy.

policy	cost $P(T)$	closed gap [95% CI]	TTT sgm (s)	reached 1%
Random (expected)	0.555	-0.20 [-0.45, -0.00]	16.7	10%
Best default solver	0.570	-0.32 [-0.72, +0.01]	15.0	16%
Best fixed strategy	0.530	+0.00 [+0.00, +0.00]	14.4	18%
Hand rules	0.530	+0.01 [-0.24, +0.19]	14.9	17%
LightGBM	0.500	+0.24 [-0.09, +0.48]	14.1	18%
MLP	0.522	+0.07 [-0.23, +0.29]	15.3	15%
Laya zero-shot	0.577	-0.37 [-0.92, -0.03]	15.6	15%
MIP-Laya (fine-tuned)	0.491	+0.31 [+0.14, +0.48]	14.4	18%
Oracle (VBS)	0.406	+1.00 [+1.00, +1.00]	13.0	21%

Table 13: Strategy selection, pooled 5-fold evaluation, $T = 10$ s, $n = 163$ test instances. Closed gap is relative to the hindsight single best strategy on the same instances (H1; 0 = SBS, 1 = oracle). TTT: time to 1% primal gap, censored at $2T$, shifted geometric mean.

G Experiments without a positive result

B0: node-selection, heuristic and cut schedules at 30 s. Five static SCIP settings (default, depth-first, best-first, aggressive heuristics, separation off) and seven schedules switching between them at the first incumbent or at 25/50 % of the budget, 45 MIPLIB instances, two seeds (Table 20). Separation off was best (9 % better than the default on P); the held-out oracle over all twelve arms added 3.2 % (P) and 5.8 % (PDI) over it, and almost none of that came from the schedules (the static oracle alone gives 3.2 % and 5.1 %).

B1: the same contrast at 120 s on pinned cores. Separation off still leads on P , but its advantage decays with the budget (20 % at 30 s, $p = 0.009$; 17 % at 60 s; 5 % at 120 s, not significant) and turns negative on PDI by 120 s as cuts pay for the bound; switching separation back on at 25 or 50 % of the budget changes nothing (Table 21).

Restarting unpromising runs. From B2 traces we estimated, by splicing recorded trajectories (a trace-based counterfactual, not a re-run), a controller that at time τ either continues the default run or restarts into another static setting for the remaining budget (incumbent and bound carried over; conservative, since a real restart also keeps presolve information). A hindsight oracle gains 5.8 % (PDI) at $\tau = 10$ s and 3.5 and 2.3 % at 20 and 30 s; a LightGBM policy on simple state features (gap, bound movement, node rate, incumbent age) did 5–8 % *worse* than never restarting (90 runs on 45 instances).

policy	cost $P(T)$	closed gap [95% CI]	TTT sgm (s)	reached 1%
Random (expected)	0.426	-0.33 [-0.81, -0.06]	39.3	18%
Best default solver	0.407	-0.14 [-0.27, -0.05]	27.5	42%
Best fixed strategy	0.393	+0.00 [+0.00, +0.00]	24.5	50%
Hand rules	0.397	-0.04 [-0.38, +0.23]	28.9	39%
LightGBM	0.389	+0.03 [-0.34, +0.30]	29.4	37%
MLP	0.393	-0.00 [-0.41, +0.26]	31.1	34%
Laya zero-shot	0.449	-0.57 [-1.20, -0.17]	36.7	22%
MIP-Laya (fine-tuned)	0.383	+0.10 [-0.20, +0.34]	26.8	44%
Oracle (VBS)	0.294	+1.00 [+1.00, +1.00]	23.4	44%

Table 14: Strategy selection, pooled 5-fold evaluation, $T = 30$ s, $n = 163$ test instances. Closed gap is relative to the hindsight single best strategy on the same instances (H1; 0 = SBS, 1 = oracle). TTT: time to 1% primal gap, censored at $2T$, shifted geometric mean.

policy	cost $P(T)$	closed gap [95% CI]	TTT sgm (s)	reached 1%
Random (expected)	0.328	-0.73 [-1.16, -0.42]	42.4	12%
Best default solver	0.299	-0.31 [-0.48, -0.17]	29.1	45%
Best fixed strategy	0.278	+0.00 [+0.00, +0.00]	26.0	52%
Hand rules	0.299	-0.30 [-0.68, -0.05]	31.2	40%
LightGBM	0.305	-0.40 [-0.71, -0.18]	30.8	40%
MLP	0.307	-0.42 [-0.77, -0.19]	31.2	40%
Laya zero-shot	0.372	-1.38 [-2.06, -0.88]	43.8	16%
MIP-Laya (fine-tuned)	0.280	-0.02 [-0.08, +0.03]	33.4	34%
Oracle (VBS)	0.210	+1.00 [+1.00, +1.00]	25.1	50%

Table 15: Strategy selection, leave-one-family-out evaluation, $T = 30$ s, $n = 313$ test instances. Closed gap is relative to the hindsight single best strategy on the same instances (H1; 0 = SBS, 1 = oracle). TTT: time to 1% primal gap, censored at $2T$, shifted geometric mean.

Rule agent. Rules distilled from the MIPLIB component results (separation off at the start, one restart at 10s without an incumbent, inference branching when the bound stalls) did not transfer to the held-out instances (-2.6% on P , not significant).

B3: switching at 300 s. The B2 design (12 arms, two seeds) on the 20 MIPLIB instances with the largest B2 headroom, at 300 s. On P the static choice is worth $+20.4\%$ [1.6, 35.9] and switching adds -0.2% [-1.8, 1.0] (at 120s on the same instances: $+13.8\%$ and -0.1%). On PDI the static choice is worth $+11.3\%$ and switching adds $+1.5\%$ [-0.2, 4.3] (120s: $+10.2\%$ and $+1.4\%$); the best schedule by mean (pseudo-cost branching for the first quarter, then default, 0.313) is within 1% of the best static setting (inference branching, 0.316). Separation off does not help at 300 s, and the online bandit is within 2% of both the default and the best static setting. One run (aggressive heuristics on opm2-z10-s4) exceeded SCIP’s 300 s limit by more than 200 s and is scored as a failure.

Exchanging incumbents between SCIP and cuOpt. SCIP (one Cortex-X925 core) and cuOpt (GPU and five Cortex-A725 cores) were launched together on the 80 MIPLIB instances (60s) in three arms: no exchange (a race), cuOpt’s incumbents handed to SCIP, and exchange in both directions (cuOpt accepts outside solutions through a set-solution callback, which disables its presolve). Time runs from the launch of both processes; the pair’s value is the better of the two incumbents at each moment. Solutions pass into SCIP through a SCIP primal heuristic called

T	policy	MIPLIB 2017	ML4CO item placement	ML4CO load balancing	PGLib-UC
10	Best fixed strategy	-0.03	+0.00	-1.77	-4.90
10	Hand rules	+0.08	-0.48	-1.90	-5.88
10	LightGBM	+0.23	-0.08	+0.12	-5.88
10	MLP	-0.04	+0.00	+0.08	-5.88
10	Laya zero-shot	+0.08	-5.26	-2.18	-5.47
10	MIP-Laya (fine-tuned)	+0.30	+0.00	+0.07	-4.84
10	<i>oracle headroom</i>	32%	17%	25%	1%
30	Best fixed strategy	+0.00	+0.00	-1.49	+0.00
30	Hand rules	+0.10	-3.21	-1.57	+0.09
30	LightGBM	+0.05	-0.80	-0.23	+0.00
30	MLP	+0.18	+0.00	+0.00	-1.81
30	Laya zero-shot	-0.06	-9.31	-0.66	-1.81
30	MIP-Laya (fine-tuned)	+0.07	+0.00	+0.15	+0.00
30	<i>oracle headroom</i>	40%	14%	23%	6%
60	Best fixed strategy	-0.01	–	–	-1.16
60	Hand rules	+0.02	–	–	+0.53
60	LightGBM	-0.26	–	–	-0.66
60	MLP	+0.08	–	–	-0.12
60	<i>oracle headroom</i>	41%	–	–	11%

Table 16: Closed gap per family, pooled policy (one model for all families). Headroom = oracle improvement over the family’s hindsight SBS. Where the headroom is small (PGLib-UC at 10s: 1%) closed-gap values are ratios of tiny differences and very large in magnitude; read them with the headroom row.

before every node and during the root LP loop, which hands a better outside solution to `trySol`: no run failed, and 38 instances received solutions. The race, with both arms interleaved in one session, beats SCIP alone by 21.5 % (44 instances, better on 28, worse on 13); one-way exchange improves SCIP’s own trajectory by 14.1 % ($p = 2 \cdot 10^{-4}$, 47 better, 24 worse) and leaves the pair’s value unchanged (+1.1 %, $p = 0.57$); two-way exchange costs 14.7 % ($p = 0.002$). Injecting solutions from SCIP’s event handler instead is not safe in this PySCIPOpt version: SCIP’s process died without a result on 9 of 80 instances per exchange arm. Solutions must be mapped between the solvers’ variable orders outside the timed window (parsing the model a second time inside SCIP’s process costs about 0.7s of the budget), and a process that dies without output is counted as a failure.

An evolutionary partner for SCIP. A population-based partner on one Cortex-A725 core exchanged solutions with SCIP (one Cortex-X925 core) in both directions: SCIP’s incumbents enter the population; offspring (uniform crossover on the integer variables, mutation, a greedy violation repair, and for mixed problems an LP over the continuous variables with the integers fixed) that improve on SCIP’s incumbent are handed to SCIP, which checks and accepts them between nodes. A second arm let up to 20 % of the population be infeasible individuals whose objective beats the best feasible one, kept with a probability that falls with their infeasibility rank (the least infeasible survive most often). On the 80 MIPLIB instances at 60s no improvement of SCIP’s primal integral was detected in either arm (−7.5 % and −3.7 %, not significant, counting three runs in which SCIP’s process died as failures; without them −0.1 % and 0.0 %), the infeasible share did not differ from the feasible-only population, and both lost to the equal-compute control of a second SCIP with another seed ($p \leq 0.015$). Of 129 improvements of its own population, only

family	n	SBS	P	oracle	best CPU GPU pair	P	best switch	P
MIPLIB 2017	80	H1	0.337	0.199	H1 C0	0.220	C0→H1@25%	0.290
ML4CO item placement	100	H1	0.207	0.174	H1 C0	0.197	H1→C0@50%	0.231
ML4CO load balancing	100	S0	0.067	0.057	S0 C0	0.059	C0→H1@25%	0.086
PGLib-UC	33	C0	0.640	0.572	H1 C0	0.568	H1→C0@25%	0.682
All	313	H1	0.254	0.185	H1 C0	0.199	H1→C0@50%	0.256

Table 17: Offline portfolios from the recorded trajectories (seed 0, full budget). CPU||GPU: one CPU strategy and one cuOpt strategy run at the same time, best incumbent of the two. Switch A→B@ x : A for x of the budget, then B from scratch (no solution transfer). The pair uses one CPU thread plus the GPU, so it is not a free lunch; it is the concurrency the DGX Spark offers.

family	n	features	med (ms)	p90 (ms)	max (ms)	overhead (own budget)
MIPLIB 2017	80		53	648	5192	1.17%
ML4CO item placement	125		16	18	20	0.25%
ML4CO load balancing	125		431	636	717	2.31%
PGLib-UC	56		1977	3338	3813	5.66%

Table 18: Policy overhead: static feature extraction (HiGHS reader + NumPy, single thread) plus one Laya decision, as a share of the family’s budget (30s ML4CO, 60s MIPLIB and PGLib-UC), using the p90 extraction time. Extraction time is dominated by parsing the MPS file, which the chosen solver repeats; a controller that handed its parsed model to the solver would pay only the NumPy part. Laya adds 56 ms per decision (median of 50 decisions, one question, batch 1).

11 beat SCIP’s incumbent at the time; SCIP accepted all 11. The partner’s feasibility test is per row.

Cheaper evaluation by racing. Replaying an F-Race-style race [6] over B0, with instances ordered by how strongly they separated the Track A strategies, reached the same verdict with 65 % of the runs (random order: 92 %). Racing selects the best single configuration; it is the wrong tool for measuring per-instance headroom, which B2 instead evaluated in instance blocks with a sequential confidence-interval stop.

H Agent-in-the-loop details

Interface. SCIP calls the agent from an event handler every Δ seconds with the current state (incumbent, bound, primal-dual gap, incumbent count and age, node count and rate, LP iterations, current setting, history). The agent returns nothing, a setting, or a restart. SCIP’s limit is wall clock (`timing/clocktype=2`; verified by sleeping inside the callback), so decision time is charged to the run; measured decision time was below 0.1 ms for both agents.

Bandit. UCB1 over the six static B2 settings; reward per interval = relative reduction of the primal-dual gap plus 0.5 for a new incumbent; $\Delta = 10$ s (set 1) or 5 s (set 2). On item placement it spent about a third of its time in aggressive heuristics, the best fixed setting for that family.

On set 2 (30 further held-out ML4CO instances (validation split), two seeds, 120s), the bandit improved P over the default by 20.8 % ($\Delta = 10$ s) and 27.1 % ($\Delta = 5$ s, $p = 3 \cdot 10^{-4}$); against the best fixed setting for the family it was -2.5 % at 120 s (not significant) and -12.5 % at 60 s

condition	runs	$P(30)$	ΔP vs A	GPU util	SM MHz	Laya ms (med/p90)
A: cuOpt alone	10	0.226	$+0.000 \pm 0.025$	71%	2405	–
B: + Laya loaded, idle	10	0.240	$+0.014 \pm 0.062$	75%	2398	–
C: + Laya every 5 s	10	0.235	$+0.009 \pm 0.028$	74%	2398	30 / 50
D: + Laya continuous	10	0.452	$+0.226 \pm 0.094$	89%	2390	26 / 44

Table 19: GPU contention between cuOpt (C0, 30 s) and Laya inference on the shared GB10 GPU. ΔP : paired difference in primal integral against condition A on the same instance (mean $\pm$ s.d. over instances and repetitions).

policy	$P(30)$	vs best static	PDI(30)	vs best static
SCIP default	0.386	-9.4%	0.539	-4.7%
best static setting	0.353	+0.0%	0.515	+0.0%
best single schedule	0.359	-1.8%	0.515	+0.0%
per-instance best static (oracle)	0.341	+3.2%	0.488	+5.1%
per-instance best incl. schedules (oracle)	0.341	+3.2%	0.485	+5.8%

Table 20: Track B go/no-go (B0): SCIP 10 with five static settings and seven fixed mid-solve schedules on 45 non-trivial MIPLIB instances, 30 s, two seeds. Every choice is made on one seed and scored on the other. PDI: primal-dual integral. Positive = better than the best static setting.

($p = 0.047$) on item placement.

Diagnosis first move. Six static settings were run on 60 ML4CO training instances (120 s); LightGBM cost regressors on the static features then predicted a setting for each of the 30 set-2 test instances, scored exactly on the recorded test runs. Primal integral $P(120)$ relative to the default: best fixed setting from training (aggressive heuristics) +28.5 % ($p = 4 \cdot 10^{-5}$), per-family majority +28.3 %, LightGBM per-instance diagnosis +23.4 %, bandit +20.3 % / +26.5 % ($\Delta = 10$ / 5 s), diagnose-then-bandit (prediction as the first move, bandit afterwards) +29.1 % ($p = 0.01$), per-run static oracle +34.8 %. On the primal-dual integral the fixed setting also led (+2.9 % vs +0.8 % for LightGBM). Laya, asked the same six-way question on a text rendering of the same features and fine-tuned on the 60 training instances (soft targets, three seeds), reached +28.3 % and, on PDI, chose aggressive heuristics on all 30 instances, i.e. the fixed setting’s +2.9 %: it learned the family rule (aggressive heuristics for item placement) and so tied the best fixed setting while beating LightGBM; zero-shot it always chose the default. Paired Wilcoxon tests over instances (seeds averaged within an instance) throughout; all 483 agent and fixed-setting solutions from these runs pass the checker.

I Resources per experiment

J Limitations

- **One machine, budgets of 30–300 s.** The cross-solver study uses 30–60 s, the SCIP studies 30–300 s; the best fixed strategy changes with the budget, so conclusions are budget-specific. ML4CO uses 75–100 training and 25 test instances per family instead of 1000/100. The whole study used one GB10 and cuOpt was the throughput bottleneck.

arm	$P(T)$			$PDI(T)$		
	30 s	60 s	120 s	30 s	60 s	120 s
SCIP default	0.286	0.193	0.126	0.466	0.343	0.249
separation off	0.229	0.160	0.119	0.428	0.336	0.269
off $\rightarrow$ default at 25 %	0.231	0.160	0.118	0.430	0.337	0.268
off $\rightarrow$ default at 50 %	0.230	0.161	0.119	0.429	0.336	0.269

Table 21: Track B, B1: SCIP 10 at 120 s on the 30 B0 instances where these arms differed most, one run per Cortex-X925 core, 8 GiB per run, no other load; 30/60 s views cut from the same runs. Paired Wilcoxon tests in the text.

metric	instances	default	bandit agent	rules agent	2 $\times$ default (2 cores)
P	ML4CO (n=40)	0.156	0.122 (+21%*)	0.160 (-2%)	0.118 (+24%*)
P	MIPLIB (n=30)	0.390	0.383 (+2%)	0.401 (-3%)	0.374 (+4%*)
P	ALL (n=70)	0.256	0.234 (+9%*)	0.263 (-3%)	0.228 (+11%*)
PDI	ML4CO (n=40)	0.513	0.508 (+1%)	0.513 (+0%)	0.495 (+3%*)
PDI	MIPLIB (n=30)	0.478	0.475 (+0%)	0.482 (-1%)	0.469 (+2%*)
PDI	ALL (n=70)	0.498	0.494 (+1%)	0.499 (-0%)	0.484 (+3%*)

Table 22: Main-experiment pilot: agents start from SCIP’s defaults in a fixed slice (one Cortex-X925 core, 9 GiB, 120 s wall clock including every agent decision). Mean over (instance, seed); percentages vs default, * paired Wilcoxon $p < 0.05$. The last column uses twice the compute and is a reference, not a competitor.

- **Concurrency is part of the measurement.** Twelve single-threaded CPU runs and two cuOpt runs shared the machine. This makes absolute times noisier and slower than on an idle machine, but it applies to every strategy and is also the regime in which a CPU||GPU pair would run. For about 11 minutes an analysis job of ours added CPU load; the 172 affected CPU runs were repeated (mean $P(60)$ changed from 0.197 to 0.195) and 43 cuOpt runs in that window were kept.
- **Heterogeneous cores.** The GB10 has ten Cortex-X925 (3.9 GHz) and ten Cortex-A725 (2.8 GHz) cores. Solver runs were not pinned, and a pinned check found SCIP explores about twice as many nodes per second on an X925 as on an A725 (4,686 vs 2,413 nodes in 20 s). Placement was random, so no strategy is favoured, but every run had an effective budget of roughly 1 $\times$ or 2 $\times$; this inflates run-to-run noise and part of the measured oracle headroom (the held-out-seed scoring of Table 11 absorbs some of it). Later experiments pin one run per fast core.
- **Single seed for the main grid.** Labels come from seed 0. The variance subset (12 MIPLIB instances, HiGHS/SCIP, five seeds) shows the oracle headroom survives noise (Table 11), but cuOpt’s own variance was not measured and GPU heuristics are typically nondeterministic.
- **Failures are scored as failures.** Three runs ignored the wall-clock limit (HiGHS on `s100`, cuOpt C1 on `sorre113`) and were killed at 150 s with no trajectory kept; they count as $P = 1$. PGLib-UC references are the best value found by any run, so UC gaps are relative to our own best, not to proven optima.
- **Six strategies, not tuned.** The portfolio is the spec’s milestone set; tuned or more diverse strategies would change the headroom. PGLib-UC has only three base systems, so “split by

family	aggr. heuristics set	$P(120)$	1st sol. (s)	zero-obj. (s)	heur. (s)	nodes
item placement	before presolve	0.155	0.01	0.01	22.5	22114
item placement	at first call	0.137	0.02	0.00	24.4	23690
item placement	after 10 s	0.152	0.02	0.00	24.5	24498
load balancing	before presolve	0.035	1.25	0.59	96.1	1
load balancing	at first call	0.031	0.73	0.00	96.0	1
load balancing	after 10 s	0.032	0.74	0.00	95.9	1

Table 23: When aggressive heuristics are applied: 30 held-out ML4CO instances (validation split), one seed, 120 s, all arms in one session, both LLM servers idle. Means per family. *1st sol.*: time of the first incumbent; *zero-obj.*: time in SCIP’s zero-objective heuristic; *heur.*: time in all primal heuristics (SCIP’s statistics); *after 10 s*: at the first call after 10 s. Nodes: branch-and-bound nodes solved.

experiment	CPU lanes	core pinning	GPU	memory per run	other load
Track A grid	12×1 thread	none (fast + efficiency cores)	2 cuOpt lanes (GPU + 3 threads)	no hard cap (SCIP limit 6 GB)	none (LLM servers stopped)
Variance subset, 300 s screening	12×1 thread	none	shared with Track A	as above	none
Laya fine-tuning	—	—	exclusive (peak 15.4 GB)	—	none
GPU contention	cuOpt 3 threads	none	1 cuOpt lane + Laya	—	none
Track B B0 (30 s)	10×1 thread	none	idle	no hard cap (SCIP 6 GB)	two LLM servers resident
Track B B1 (120 s)	10×1 thread	one run per Cortex-X925 core	idle	8 GiB hard cap (SCIP 7 GB)	none (LLM servers stopped)
B2 (120 s), extra seeds	10×1 thread	one run per Cortex-X925 core	idle	9 GiB hard cap	none (LLM servers stopped)
B3 (300 s)	9×1 thread	one run per Cortex-X925 core	idle	9 GiB hard cap	none (LLM servers stopped)
Agent pilots, diagnosis	10×1 thread	one run per Cortex-X925 core	idle	9 GiB hard cap	none (LLM servers stopped)
LLM agents, timing controls	4×1 thread	one run per Cortex-X925 core	local LLM server	8 GiB hard cap	local LLM server (the agent)
Timing mechanism test	3×1 thread	one run per Cortex-X925 core	idle	4 GiB hard cap	both LLM servers resident, idle
Live pairs, night (race, exchange, evolutionary)	SCIP 1 thread	SCIP on one Cortex-X925 core	cuOpt + five Cortex-A725 cores	9 GiB hard cap (SCIP)	none (LLM servers stopped)
Live pairs, day (heuristic exchange, HIGHS pair, same-session race)	SCIP/HIGHS 1 thread	one Cortex-X925 core	cuOpt + five Cortex-A725 cores	9 GiB hard cap (SCIP)	local LLM server resident, idle

Table 24: Compute allocated to each experiment on the DGX Spark (10 Cortex-X925 at 3.9 GHz + 10 Cortex-A725 at 2.8 GHz, 121 GiB unified memory). SCIP explores about twice as many nodes per second on an X925 core as on an A725 core, so unpinned runs had a random effective budget.

system” has three groups.

- **Laya input is a text rendering of 62 static features.** It sees no solver state; a richer state (root LP statistics, early incumbents) is where a text-encoder policy could differ most from LightGBM and was not tested. Fine-tuning hyper-parameters were set once and not tuned, and each fold was fine-tuned with a single seed. No superiority of fine-tuned Laya over LightGBM was established on the families seen in training (paired $p = 0.67$ at 10 s); on a family left out of training Laya was better at 30 s.
- **Pairs: offline and live.** The cross-solver pair estimate is computed from trajectories recorded under concurrent load; the live SCIP||cuOpt run with both arms in one session gives 21.5 % on 44 of the 80 instances; comparisons across sessions would fold in session drift. Solution exchange was tested for one solver pair and one budget (60 s).

K Companion file for automated readers

A single markdown file, `AGENTS.md`, accompanies this paper as an arXiv ancillary file and at the root of the repository <https://github.com/berkorbay/optopt>. It lists every claim with its numbers, the tables, data files and scripts behind it, and its caveats, together with a map of the repository, reproduction commands and the rules used for new experiments, so that an AI agent can check or extend the results without parsing this PDF.

L Reproducibility and use of AI tools

Code, job files, derived tables, per-run resource records and the research journal are in the public repository <https://github.com/berkorbay/optopt>, packaged as the Python library `optopt`. The original run trajectories, stored solutions, launch and resource records are published as a versioned release archive with SHA-256 checksums and the code revision (<https://github.com/berkorbay/optopt/releases/tag/traces-v1>); from them the modules in `optopt.analysis` generate every table in this paper, and `optopt.reproduce` re-derives all of them and fails if any step fails or any table differs. Re-running the job files instead is a new replication, whose wall-clock results will differ. The experiments were designed with and executed by AgentRA (Agent Research Assistant; AgentRA is mainly Claude Opus 5.5), under the author’s direction; literature references were verified against publisher or proceedings pages before inclusion. The repository’s code, data, research vault and tutorials are released under the MIT License, and this paper under CC BY 4.0.